

Laporan Praktikum Kontrol Cerdas

Nama : Fryscha Febryanti Puspitasari
NIM : 224308008
Kelas : TKA 6A
Akun Github (Tautan) : <https://github.com/Fryscha>
Student Lab Assistant : Yulia Brilianty

1. Judul Percobaan

Reinforcement Learning for Autonomous Control

2. Tujuan Percobaan

Percobaan ini dilakukan dengan tujuan sebagai berikut.

- Melatih dan menguji agen Reinforcement Learning (RL) untuk mengontrol lingkungan secara otonom.
- Memahami konsep dasar Reinforcement Learning (RL) dalam sistem kendali.
- Mengimplementasikan agen Reinforcement Learning (RL) menggunakan algoritma Deep Q-Network (DQN).
- Menggunakan GitHub untuk version control dan dokumentasi praktikum.
- Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan Reinforcement Learning (RL).

3. Landasan Teori

Reinforcement Learning (RL) adalah paradigma pembelajaran mesin di mana sebuah agen belajar untuk membuat keputusan dengan berinteraksi dengan lingkungan untuk memaksimalkan reward kumulatif. Dalam konteks sistem kendali, RL memungkinkan agen untuk belajar mengontrol lingkungan secara otonom melalui proses coba-coba (*trial and error*). Agen menerima umpan balik dalam bentuk *reward* setelah melakukan tindakan, dan berdasarkan umpan balik ini, agen menyesuaikan strateginya (kebijakan) untuk memilih tindakan yang menghasilkan *reward* tertinggi. Proses pelatihan ini melibatkan eksplorasi

lingkungan untuk menemukan tindakan yang optimal dan eksploitasi pengetahuan yang telah dipelajari untuk memaksimalkan *reward*. Pengujian agen RL dilakukan untuk mengevaluasi kinerja agen dalam lingkungan yang telah ditentukan, memastikan bahwa agen dapat menggeneralisasi pengetahuannya dan beradaptasi dengan perubahan dalam lingkungan.

Algoritma Deep Q-Network (DQN) adalah salah satu algoritma RL yang populer, terutama dalam menangani masalah dengan ruang keadaan yang besar. DQN menggunakan jaringan saraf tiruan (*neural network*) untuk memperkirakan fungsi Q, yang memetakan pasangan keadaan-tindakan ke nilai Q (nilai harapan *reward* kumulatif). Dalam implementasinya, DQN menggunakan teknik *experience replay* untuk menyimpan pengalaman agen dalam bentuk transisi (keadaan, tindakan, *reward*, keadaan berikutnya) dan melatih jaringan saraf menggunakan sampel acak dari memori ini. Selain itu, DQN menggunakan jaringan target (*target network*) untuk menstabilkan proses pelatihan. Implementasi DQN melibatkan pemilihan arsitektur jaringan saraf yang sesuai, penentuan parameter pelatihan (seperti laju pembelajaran dan faktor diskon), dan pemilihan strategi eksplorasi-eksploitasi (seperti epsilon-greedy).

OpenAI Gym adalah toolkit yang menyediakan berbagai lingkungan simulasi untuk melatih dan menguji agen RL. OpenAI Gym menyediakan antarmuka yang mudah digunakan untuk berinteraksi dengan lingkungan, memungkinkan pengembang untuk fokus pada implementasi algoritma RL tanpa harus khawatir tentang detail implementasi lingkungan. Penggunaan OpenAI Gym memungkinkan eksperimen yang terkontrol dan reproduktif, serta memfasilitasi perbandingan kinerja berbagai algoritma RL. Gymnasium adalah versi terbaru dan terus dikelola dari OpenAI Gym, menyediakan interface standar untuk lingkungan Reinforcement Learning.

GitHub digunakan sebagai platform untuk version control dan dokumentasi proyek. Version control memungkinkan pengembang untuk melacak perubahan dalam kode dan berkolaborasi dengan orang lain secara efektif. Dokumentasi proyek mencakup deskripsi algoritma, implementasi kode, dan hasil eksperimen. Penggunaan GitHub memfasilitasi transparansi dan reproduktibilitas penelitian, serta memungkinkan orang lain untuk membangun di atas pekerjaan yang telah dilakukan. GitHub adalah toolkit untuk mengembangkan dan membandingkan algoritma

Python adalah salah satu bahasa pemrograman yang digunakan dalam pengembangan sistem kontrol cerdas. Bahasa pemrograman ini sangat diminati oleh pengguna karena kemudahan penggunaannya dan banyaknya pustaka yang tersedia.

4. Analisis dan Diskusi

Analisis Hasil:

- a. Bagaimana performa agen dalam mengontrol environment CartPole?

Jawab:

Agen yang diimplementasikan menggunakan algoritma Deep Q-Network (DQN) menunjukkan performa yang cukup baik dalam mengontrol environment CartPole. Dalam pengujian, agen mampu mempertahankan tiang dalam posisi tegak selama beberapa episode, dengan skor yang meningkat seiring dengan bertambahnya episode. Skor maksimum yang dicapai oleh agen dalam pengujian adalah jumlah langkah yang berhasil diambil sebelum tiang jatuh. Dengan pengaturan epsilon yang rendah (0.01) selama pengujian, agen lebih cenderung untuk mengeksploitasi pengetahuan yang telah dipelajari daripada melakukan eksplorasi, yang menunjukkan bahwa agen telah belajar strategi yang efektif untuk mengontrol tiang.

- b. Bagaimana perubahan parameter (misal: gamma, epsilon, learning rate) mempengaruhi kinerja agen?

Jawab:

Perubahan parameter seperti gamma, epsilon, dan learning rate memiliki dampak signifikan terhadap kinerja agen.

Gamma: Parameter ini menentukan seberapa jauh agen mempertimbangkan reward masa depan. Nilai gamma yang lebih tinggi (mendekati 1) membuat agen lebih fokus pada reward jangka panjang, sedangkan nilai yang lebih rendah membuat agen lebih fokus pada reward jangka pendek. Dalam konteks CartPole, gamma yang lebih tinggi dapat membantu agen belajar strategi yang lebih baik untuk menjaga tiang tetap tegak.

Epsilon: Parameter ini mengontrol tingkat eksplorasi agen. Nilai epsilon yang tinggi memungkinkan agen untuk lebih banyak mengeksplorasi tindakan yang berbeda, yang penting pada tahap awal pelatihan. Namun, saat pelatihan berlangsung, epsilon harus

dikurangi untuk meningkatkan eksploitasi. Jika epsilon terlalu rendah terlalu cepat, agen mungkin tidak menemukan strategi optimal.

Learning Rate: Learning rate menentukan seberapa besar perubahan yang dilakukan pada bobot model berdasarkan kesalahan yang dihitung. Learning rate yang terlalu tinggi dapat menyebabkan model tidak stabil, sedangkan yang terlalu rendah dapat memperlambat proses pelatihan. Penyesuaian learning rate yang tepat sangat penting untuk mencapai konvergensi yang baik.

c. Apa tantangan yang muncul selama pelatihan agen RL?

Jawab:

Beberapa tantangan yang muncul selama pelatihan agen RL antara lain:

Stabilitas Pelatihan: Pelatihan agen RL sering kali tidak stabil, terutama ketika menggunakan jaringan saraf. Fluktuasi dalam reward dan pembaruan bobot dapat menyebabkan model tidak konvergen.

Eksplorasi vs. Eksploitasi: Menemukan keseimbangan antara eksplorasi dan eksploitasi adalah tantangan utama. Terlalu banyak eksplorasi dapat menghambat pembelajaran, sementara terlalu banyak eksploitasi dapat menyebabkan agen terjebak dalam strategi suboptimal.

Pengaturan Hyperparameter: Memilih hyperparameter yang tepat (seperti gamma, epsilon, dan learning rate) sangat penting dan sering kali memerlukan eksperimen yang ekstensif.

Keterbatasan Memori: Agen DQN menggunakan experience replay, yang memerlukan memori untuk menyimpan pengalaman. Keterbatasan memori dapat membatasi ukuran batch yang dapat digunakan untuk pelatihan.

Diskusi:

a. Apa perbedaan utama antara Reinforcement Learning dan metode supervised learning dalam sistem kendali?

Jawab:

Perbedaan utama antara Reinforcement Learning (RL) dan metode supervised learning terletak pada cara agen belajar dari data. Dalam supervised learning, model dilatih menggunakan dataset yang berisi pasangan input-output yang benar, di mana tujuan model adalah memprediksi output yang benar untuk input yang diberikan. Sebaliknya,

dalam RL, agen belajar melalui interaksi dengan lingkungan, di mana tidak ada pasangan input-output yang eksplisit. Agen menerima umpan balik dalam bentuk reward atau penalty berdasarkan tindakan yang diambil, dan tujuan agen adalah memaksimalkan reward kumulatif. Ini membuat RL lebih cocok untuk masalah di mana keputusan harus diambil secara berurutan dan hasil dari tindakan tidak selalu langsung terlihat.

- b. Bagaimana strategi eksplorasi (exploration) dan eksploitasi (exploitation) dapat dioptimalkan pada agen RL?

Jawab:

Strategi eksplorasi dan eksploitasi dapat dioptimalkan dengan beberapa pendekatan:

Epsilon Decay: Mengurangi nilai epsilon secara bertahap selama pelatihan untuk meningkatkan eksploitasi seiring waktu. Ini memungkinkan agen untuk mengeksplorasi lebih banyak di awal dan beralih ke eksploitasi saat pengetahuan meningkat.

Upper Confidence Bound (UCB): Menggunakan pendekatan berbasis statistik untuk memilih tindakan yang tidak hanya mempertimbangkan nilai Q tetapi juga ketidakpastian dalam estimasi tersebut.

Thompson Sampling: Menggunakan sampling probabilistik untuk memilih tindakan berdasarkan distribusi posterior dari nilai Q, yang memungkinkan agen untuk mengeksplorasi tindakan yang kurang diketahui.

Prioritized Experience Replay: Menggunakan pengalaman yang lebih informatif untuk pelatihan, sehingga agen dapat belajar lebih cepat dari pengalaman yang lebih berharga.

- c. Potensi aplikasi lain dari RL dalam sistem kendali nyata apa saja yang dapat diimplementasikan?

Jawab:

Reinforcement Learning memiliki potensi aplikasi yang luas dalam sistem kendali nyata, antara lain:

Robotika: Mengontrol robot untuk melakukan tugas-tugas kompleks seperti navigasi, pengambilan objek, dan interaksi dengan manusia.

Sistem Kendali Otomatis: Mengoptimalkan kendali dalam sistem seperti HVAC (Heating, Ventilation, and Air Conditioning) untuk efisiensi energi.

Pengendalian Kendaraan Otonom: Mengembangkan algoritma untuk mengendalikan kendaraan otonom dalam berbagai kondisi lalu lintas dan cuaca.

Permainan dan Simulasi: Menerapkan RL untuk mengembangkan agen yang dapat bermain game atau berinteraksi dalam simulasi dengan cara yang lebih manusiawi.

Sistem Keuangan: Menggunakan RL untuk mengoptimalkan strategi perdagangan dan manajemen portofolio dalam pasar keuangan.

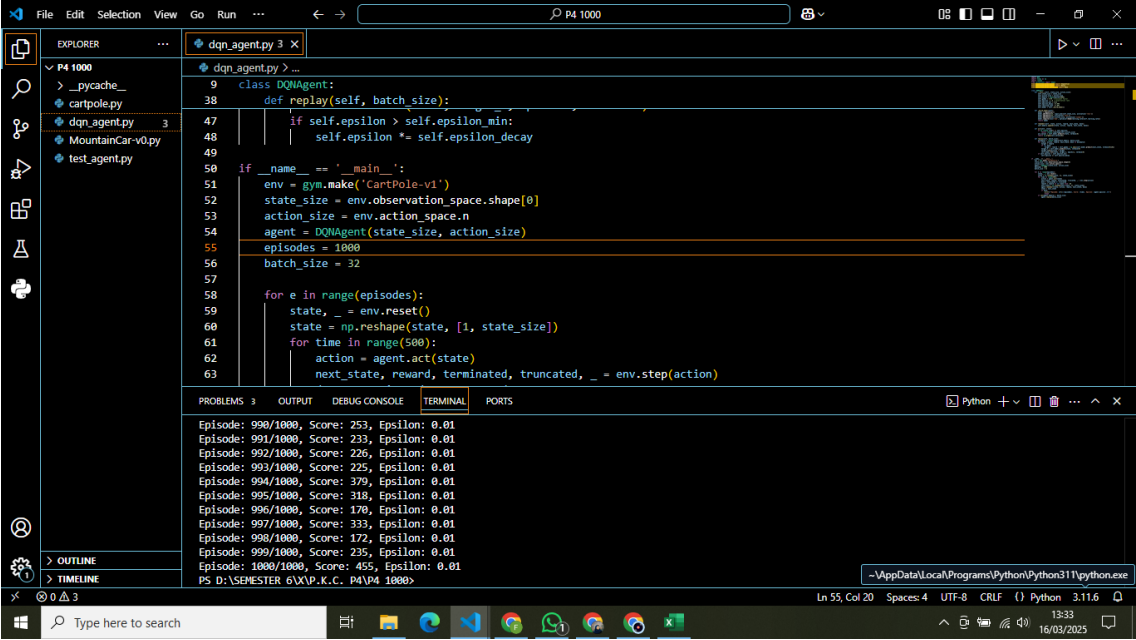
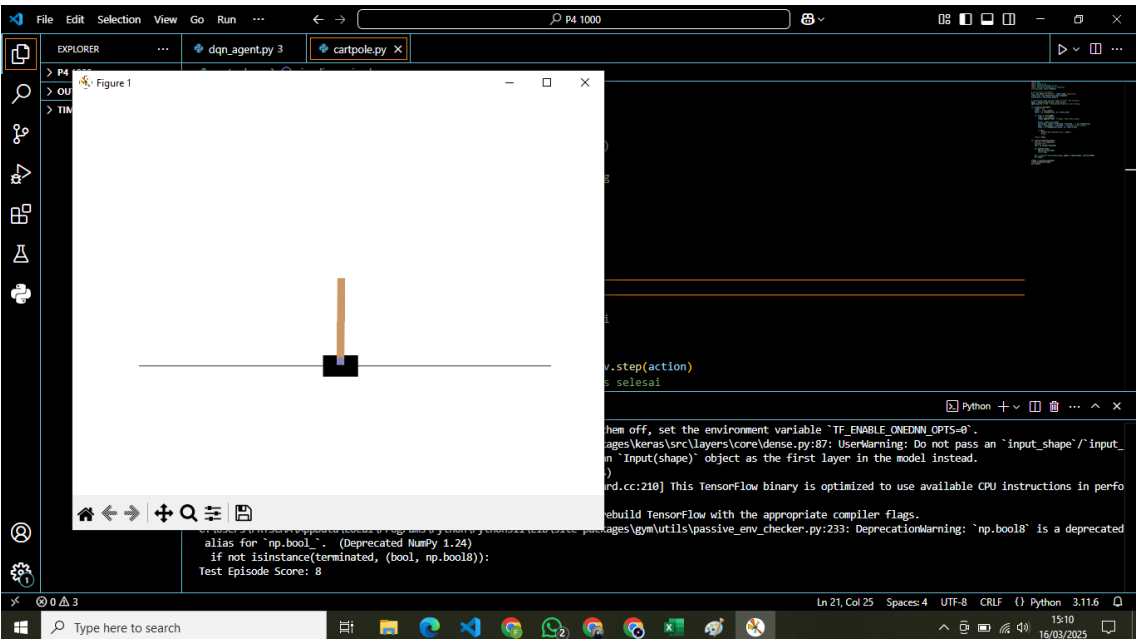
5. Assignment

Topik percobaan ini adalah memodifikasi algoritma DQN dengan menambahkan teknik seperti target network untuk meningkatkan stabilitas pelatihan. Selain itu, melakukan uji agen pada environment lain seperti LunarLander-v2 atau MountainCar-v0 dan membandingkan performanya.

Hasil percobaan yang telah dilakukan yaitu pada percobaan memodifikasi algoritma DQN dengan menambahkan teknik seperti target network untuk meningkatkan stabilitas pelatihan dapat disimpulkan bahwa semakin besar episode yang dilatih, maka semakin bagus performa agen dalam mengontrol environment CartPole. Kemudian, saya menggunakan uji agen pada environment MountainCar-v0. Saya bandingkan keduanya dapat saya simpulkan bahwa MountainCar-v0 lebih membutuhkan episode lebih banyak daripada CartPole agar dapat memproses performanya. Contoh kasus pada CartPole hanya menggunakan 1000 episode mampu untuk memproses performa sampai selesai dan hasil output yang dihasilkan sesuai dengan perintah. Sedangkan pada MountainCar-v0 hanya 1000 episode masih belum cukup untuk memproses performa sampai output yang dihasilkan sesuai perintah. MountainCar-v0 membutuhkan lebih dari 1000 episode hingga performa bisa menyelesaikan dan menghasilkan output sesuai perintah. Akan tetapi, hal ini dapat mengakibatkan perangkat cepat rusak.

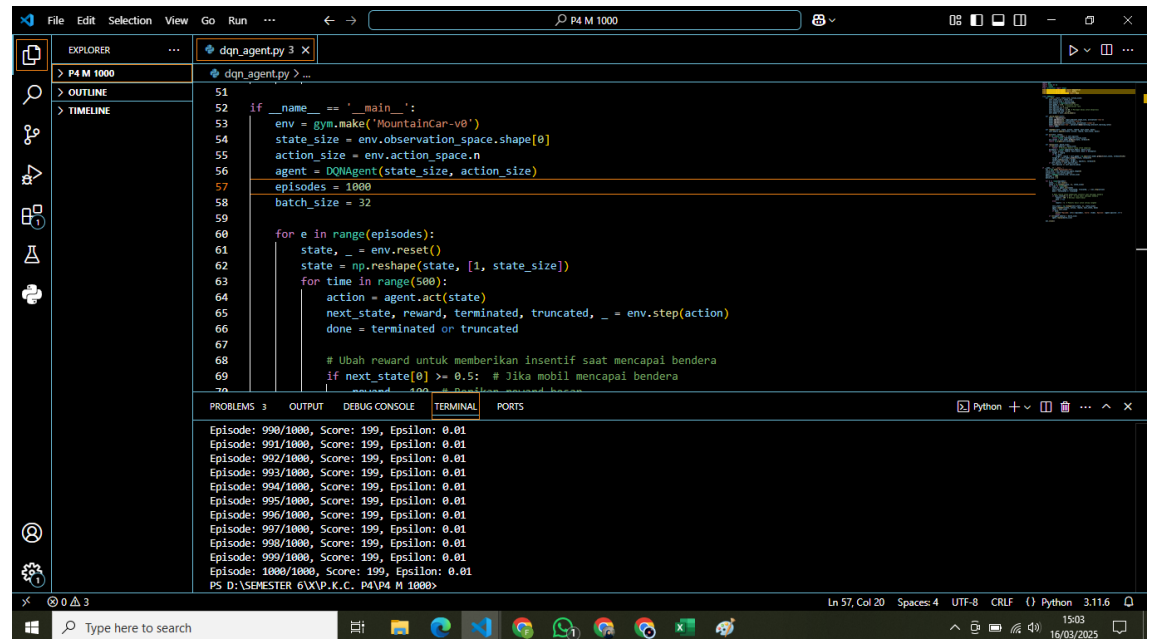
6. Data dan Output Hasil Pengamatan

Sajikan data dan hasil yang diperoleh selama percobaan. Gunakan tabel untuk menyajikan data jika diperlukan.

No	Variabel	Hasil Pengamatan
1	CartPole 1000 Episode	Output yang ditampilkan memiliki rentang score yaitu 10 – 425 (semakin banyak episode, maka semakin naik nilai score) dengan nilai awal epsilon 1.0 yang semakin banyak episode, maka semakin turun nilai epsilon.  <pre> class DQNAgent: def replay(self, batch_size): if self.epsilon > self.epsilon_min: self.epsilon *= self.epsilon_decay if __name__ == '__main__': env = gym.make('CartPole-v1') state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32 for e in range(episodes): state, _ = env.reset() state = np.reshape(state, [1, state_size]) for time in range(500): action = agent.act(state) next_state, reward, terminated, truncated, _ = env.step(action) </pre> Episode: 998/1000, Score: 253, Epsilon: 0.01 Episode: 999/1000, Score: 233, Epsilon: 0.01 Episode: 1000/1000, Score: 226, Epsilon: 0.01 Episode: 993/1000, Score: 225, Epsilon: 0.01 Episode: 994/1000, Score: 379, Epsilon: 0.01 Episode: 995/1000, Score: 318, Epsilon: 0.01 Episode: 996/1000, Score: 170, Epsilon: 0.01 Episode: 997/1000, Score: 333, Epsilon: 0.01 Episode: 998/1000, Score: 172, Epsilon: 0.01 Episode: 999/1000, Score: 235, Epsilon: 0.01 Episode: 1000/1000, Score: 425, Epsilon: 0.01
2	Output CartPole	Output yang dihasilkan yaitu test episode score sebesar 8 dan menampilkan visual tiang tetap tegak.  Test Episode Score: 8

3 Mountain
Car-v0
1000
Episode

Output yang ditampilkan memiliki score yaitu 199 dengan nilai awal epsilon 1.00 yang semakin banyak episode, maka semakin turun nilai episode.

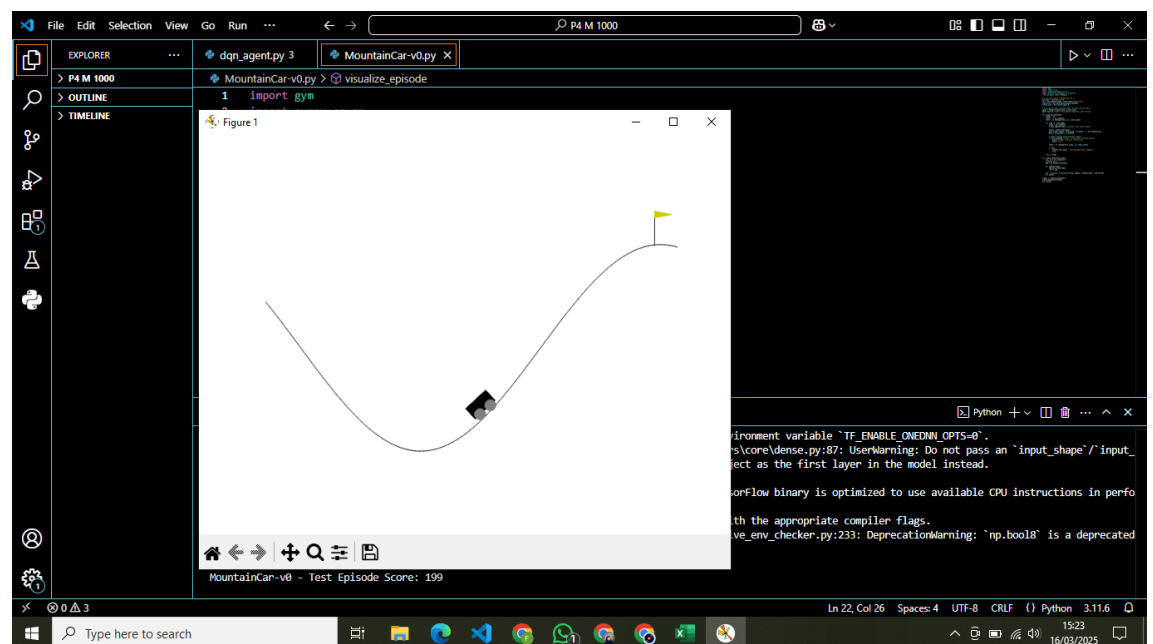


```
51
52 if __name__ == '__main__':
53     env = gym.make('MountainCar-v0')
54     state_size = env.observation_space.shape[0]
55     action_size = env.action_space.n
56     agent = DQNAgent(state_size, action_size)
57     episodes = 1000
58     batch_size = 32
59
60     for e in range(episodes):
61         state, _ = env.reset()
62         state = np.reshape(state, [1, state_size])
63         for time in range(500):
64             action = agent.act(state)
65             next_state, reward, terminated, truncated, _ = env.step(action)
66             done = terminated or truncated
67
68             # Ubah reward untuk memberikan insentif saat mencapai bendera
69             if next_state[0] >= 0.5: # Jika mobil mencapai bendera
70                 reward = 100 # Definisi reward bendera
```

Episode: 990/1000, Score: 199, Epsilon: 0.01
Episode: 991/1000, Score: 199, Epsilon: 0.01
Episode: 992/1000, Score: 199, Epsilon: 0.01
Episode: 993/1000, Score: 199, Epsilon: 0.01
Episode: 994/1000, Score: 199, Epsilon: 0.01
Episode: 995/1000, Score: 199, Epsilon: 0.01
Episode: 996/1000, Score: 199, Epsilon: 0.01
Episode: 997/1000, Score: 199, Epsilon: 0.01
Episode: 998/1000, Score: 199, Epsilon: 0.01
Episode: 999/1000, Score: 199, Epsilon: 0.01
Episode: 1000/1000, Score: 199, Epsilon: 0.01
PS D:\SEMESTER 6\XP.K.C. P4\VP4 M 1000>

4 Output
Mountain
Car-v0

Output yang dihasilkan yaitu test episode score sebesar 199 dan menampilkan visual mobil bergerak.



7. Kesimpulan

Berdasarkan percobaan dan analisis data dapat disimpulkan bahwa

- a. Reinforcement Learning (RL) adalah paradigma pembelajaran mesin di mana sebuah agen belajar untuk membuat keputusan dengan berinteraksi dengan lingkungan untuk memaksimalkan reward kumulatif.
- b. Algoritma Deep Q-Network (DQN) adalah salah satu algoritma RL yang populer, terutama dalam menangani masalah dengan ruang keadaan yang besar. DQN menggunakan jaringan saraf tiruan (*neural network*) untuk memperkirakan fungsi Q, yang memetakan pasangan keadaan-tindakan ke nilai Q (nilai harapan *reward* kumulatif).
- c. Semakin besar episode yang dilatih, maka semakin bagus performa agen dalam mengontrol environment.

8. Saran

Pemilihan agen dalam mengontrol environment harus memperhatikan kemampuan perangkat yang digunakan.

9. Daftar Pustaka

- Kim, C. (2024). Discrete Space Deep Reinforcement Learning Algorithm Based on Support Vector Machine Recursive Feature Elimination. *Symmetry*, 16(8), 940. <https://doi.org/10.3390/sym16080940>
- Muttaqin, Arafah M., Jaya A.K., Suryawan M.A., Gustiana Z., Banjarnahor A.R., Bukidz D.P., Hazriani, Simanjuntak M., Saputra N., Fajrillah. 2023. *Implementasi Artificial Intelligence (AI) dalam Kehidupan*. Cetakan ke-1. Yayasan Kita Menulis. Langsa.
- Santoso J.T., 2023. *KECERDASAN BUATAN (Artificial Intelligence)*. Yayasan Prima Agus Teknik Bekerja sama dengan Universitas Sains & Teknologi Komputer (Universitas STEKOM). Semarang.
- Siregar, M. H., & Mulyana, D. I. (2024). Teknologi Artificial Intelligence (AI) Vision Swift dalam Sistem Pemantauan Latihan Bulu Tangkis dengan Algoritma Optical Flow. *Jurnal Indonesia: Manajemen Informatika dan Komunikasi*, 5(3), 3349–3361. <https://doi.org/10.35870/jimik.v5i3.1027>

LAMPIRAN

1. Program CartPole

```
dqn_agent.py
```

```
import gym
```

```
import numpy as np
```

```
import random
```

```
from collections import deque
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.optimizers import Adam
```

```
class DQNAgent:
```

```
    def __init__(self, state_size, action_size):
```

```
        self.state_size = state_size
```

```
        self.action_size = action_size
```

```
        self.memory = deque(maxlen=2000)
```

```
        self.gamma = 0.95      # Discount factor
```

```
        self.epsilon = 1.0     # Exploration rate
```

```
        self.epsilon_min = 0.01
```

```
        self.epsilon_decay = 0.995
```

```
        self.learning_rate = 0.001
```

```
        self.model = self._build_model()
```

```
    def _build_model(self):
```

```
        model = Sequential()
```

```
        model.add(Dense(24, input_dim=self.state_size, activation='relu'))
```

```
        model.add(Dense(24, activation='relu'))
```

```
        model.add(Dense(self.action_size, activation='linear'))
```

```
        model.compile(loss='mse', optimizer=Adam(learning_rate=self.learning_rate))
```

```
return model
```

```
def remember(self, state, action, reward, next_state, done):
```

```
    self.memory.append((state, action, reward, next_state, done))
```

```
def act(self, state):
```

```
    if np.random.rand() <= self.epsilon:
```

```
        return random.randrange(self.action_size)
```

```
    act_values = self.model.predict(state, verbose=0)
```

```
    return np.argmax(act_values[0])
```

```
def replay(self, batch_size):
```

```
    minibatch = random.sample(self.memory, batch_size)
```

```
    for state, action, reward, next_state, done in minibatch:
```

```
        target = reward
```

```
        if not done:
```

```
            target = reward + self.gamma * np.amax(self.model.predict(next_state, verbose=0)[0])
```

```
        target_f = self.model.predict(state, verbose=0)
```

```
        target_f[0][action] = target
```

```
        self.model.fit(state, target_f, epochs=1, verbose=0)
```

```
    if self.epsilon > self.epsilon_min:
```

```
        self.epsilon *= self.epsilon_decay
```

```
if __name__ == '__main__':
```

```
    env = gym.make('CartPole-v1')
```

```
    state_size = env.observation_space.shape[0]
```

```
    action_size = env.action_space.n
```

```
    agent = DQNAgent(state_size, action_size)
```

```
    episodes = 1000
```

```
    batch_size = 32
```

```

for e in range(epochs):
    state, _ = env.reset()
    state = np.reshape(state, [1, state_size])
    for time in range(500):
        action = agent.act(state)
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        reward = reward if not done else -10
        next_state = np.reshape(next_state, [1, state_size])
        agent.remember(state, action, reward, next_state, done)
        state = next_state
    if done:
        print(f"Episode: {e+1}/{epochs}, Score: {time}, Epsilon: {agent.epsilon:.2f}")
        break
    if len(agent.memory) > batch_size:
        agent.replay(batch_size)

```

test_agent.py

```

import gym
import numpy as np
from dqn_agent import DQNAgent

env = gym.make('CartPole-v1')
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# Inisialisasi agen (gunakan model terlatih jika tersedia)
agent = DQNAgent(state_size, action_size)
agent.epsilon = 0.01 # Minimalkan eksplorasi saat testing

```

```

for e in range(5):
    state, _ = env.reset()
    state = np.reshape(state, [1, state_size])
    for time in range(500):
        env.render()
        action = agent.act(state)
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        state = np.reshape(next_state, [1, state_size])
        if done:
            print(f"Test Episode: {e+1}, Score: {time}")
            break
env.close()

```

CartPole.py

```

import gym
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from dqn_agent import DQNAgent

# Inisialisasi environment
env = gym.make('CartPole-v1', render_mode='rgb_array')
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# Inisialisasi agen (gunakan model terlatih jika tersedia)
agent = DQNAgent(state_size, action_size)
agent.epsilon = 0.01 # Minimalkan eksplorasi saat testing

```

```

def visualize_episode():
    frames = []
    state, _ = env.reset()
    state = np.reshape(state, [1, state_size])

    for time in range(5000):
        frame = env.render()
        frames.append(frame) # Simpan frame untuk animasi

        action = agent.act(state)
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated # Gabungkan status selesai
        state = np.reshape(next_state, [1, state_size])

    if done:
        print(f"Test Episode Score: {time}")
        break

    return frames

def create_animation(frames):
    fig, ax = plt.subplots()
    ax.axis('off')
    img = ax.imshow(frames[0])

    def update(frame):
        img.set_array(frame)
        return img,

```

```
ani = animation.FuncAnimation(fig, update, frames=frames, interval=5000)
```

```
plt.show()
```

```
frames = visualize_episode()
```

```
create_animation(frames)
```

```
env.close()
```

2. Program MountainCar-v0

```
dqn_agent.py
```

```
import gym
```

```
import numpy as np
```

```
import random
```

```
from collections import deque
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.optimizers import Adam
```

```
class DQNAgent:
```

```
    def __init__(self, state_size, action_size):
```

```
        self.state_size = state_size
```

```
        self.action_size = action_size
```

```
        self.memory = deque(maxlen=2000)
```

```
        self.gamma = 0.95 # Discount factor
```

```
        self.epsilon = 1.0 # Exploration rate
```

```
        self.epsilon_min = 0.01
```

```
        self.epsilon_decay = 0.99 # Percepat decay untuk eksplorasi
```

```
        self.learning_rate = 0.001
```

```
        self.model = self._build_model()
```

```

def _build_model(self):
    model = Sequential()
    model.add(Dense(24, input_dim=self.state_size, activation='relu'))
    model.add(Dense(24, activation='relu'))
    model.add(Dense(self.action_size, activation='linear'))
    model.compile(loss='mse', optimizer=Adam(learning_rate=self.learning_rate))
    return model

def remember(self, state, action, reward, next_state, done):
    self.memory.append((state, action, reward, next_state, done))

def act(self, state):
    if np.random.rand() <= self.epsilon:
        return random.randrange(self.action_size)
    act_values = self.model.predict(state, verbose=0)
    return np.argmax(act_values[0])

def replay(self, batch_size):
    if len(self.memory) < batch_size:
        return # Pastikan memori cukup untuk sampling
    minibatch = random.sample(self.memory, batch_size)
    for state, action, reward, next_state, done in minibatch:
        target = reward
        if not done:
            target = reward + self.gamma * np.amax(self.model.predict(next_state, verbose=0)[0])
        target_f = self.model.predict(state, verbose=0)
        target_f[0][action] = target
        self.model.fit(state, target_f, epochs=1, verbose=0)
    if self.epsilon > self.epsilon_min:
        self.epsilon *= self.epsilon_decay

```



```

if __name__ == '__main__':
    env = gym.make('MountainCar-v0')
    state_size = env.observation_space.shape[0]
    action_size = env.action_space.n
    agent = DQNAgent(state_size, action_size)
    episodes = 1000
    batch_size = 32

    for e in range(episodes):
        state, _ = env.reset()
        state = np.reshape(state, [1, state_size])
        for time in range(500):
            action = agent.act(state)
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            # Ubah reward untuk memberikan insentif saat mencapai bendera
            if next_state[0] >= 0.5: # Jika mobil mencapai bendera
                reward = 100 # Berikan reward besar
                done = True
            else:
                reward = -1 # Penalti kecil untuk setiap langkah

            next_state = np.reshape(next_state, [1, state_size])
            agent.remember(state, action, reward, next_state, done)
            state = next_state

        if done:
            print(f"Episode: {e+1}/{episodes}, Score: {time}, Epsilon: {agent.epsilon:.2f}")
            break

```

```
if len(agent.memory) > batch_size:
```

```
    agent.replay(batch_size)
```

```
env.close()
```

```
test_agent.py
```

```
import gym
```

```
import numpy as np
```

```
from dqn_agent import DQNAgent
```

```
env = gym.make('MountainCar-v0')
```

```
state_size = env.observation_space.shape[0]
```

```
action_size = env.action_space.n
```

```
# Inisialisasi agen (gunakan model terlatih jika tersedia)
```

```
agent = DQNAgent(state_size, action_size)
```

```
agent.epsilon = 0.01 # Minimalkan eksplorasi saat testing
```

```
for e in range(5): # Uji selama 5 episode
```

```
    state, _ = env.reset()
```

```
    state = np.reshape(state, [1, state_size])
```

```
    for time in range(500):
```

```
        env.render() # Menampilkan lingkungan
```

```
        action = agent.act(state) # Ambil tindakan berdasarkan model
```

```
        next_state, reward, terminated, truncated, _ = env.step(action)
```

```
        done = terminated or truncated
```

```
        state = np.reshape(next_state, [1, state_size])
```

```
    if done:
```

```
        print(f"Test Episode: {e+1}, Score: {time}")
```

```
        break
```

```
env.close()
```

```
MountainCar-v0.py
```

```
import gym
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.animation as animation
```

```
from dqn_agent import DQNAgent
```

```
# Pilih environment ('MountainCar-v0')
```

```
env_name = 'MountainCar-v0'
```

```
env = gym.make(env_name, render_mode='rgb_array')
```

```
state_size = env.observation_space.shape[0]
```

```
action_size = env.action_space.n
```

```
# Inisialisasi agen (gunakan model terlatih jika tersedia)
```

```
agent = DQNAgent(state_size, action_size)
```

```
agent.epsilon = 0.01 # Minimalkan eksplorasi saat testing
```

```
def visualize_episode():
```

```
    frames = []
```

```
    state, _ = env.reset()
```

```
    state = np.reshape(state, [1, state_size])
```

```
    for time in range(5000):
```

```
        frame = env.render()
```

```
        frames.append(frame) # Simpan frame untuk animasi
```

```
        action = agent.act(state)
```

```
        next_state, reward, terminated, truncated, _ = env.step(action)
```

done = terminated or truncated

Reward shaping untuk mencapai bendera

if next_state[0] >= 0.5: # Jika mobil mencapai bendera

 reward += 100 # Berikan reward besar

 done = True

state = np.reshape(next_state, [1, state_size])

if done:

 print(f"{env_name} - Test Episode Score: {time}")

 break

return frames

def create_animation(frames):

 fig, ax = plt.subplots()

 ax.axis('off')

 img = ax.imshow(frames[0])

def update(frame):

 img.set_array(frame)

 return img,

ani = animation.FuncAnimation(fig, update, frames=frames, interval=50)

plt.show()

frames = visualize_episode()

create_animation(frames)

env.close()